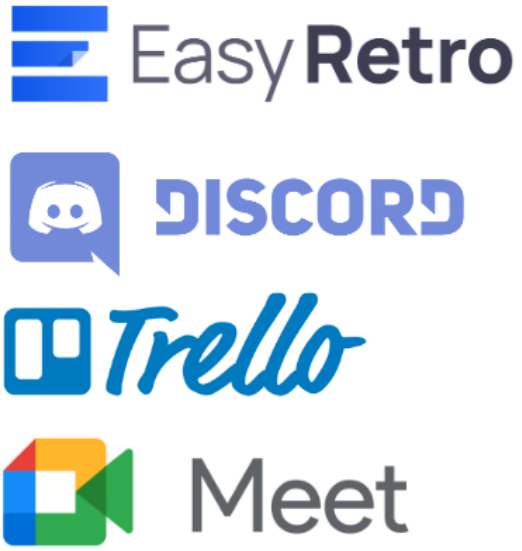
O que levou a equipe a tomar decisões de tecnologias e abordagens:



Ferramentas

Softwares utilizados para gerenciamento de **tarefas** e **realização de cerimônias**

Scrum: Modelo ágil e a equipe toda possui conhecimento.
EasyRetro: Gratuito e permite ao usuário escrever de forma anônima durante as retrôs do Scrum.
Discord: Gratuito e bastante prático para alinhamentos da equipe.
Trello: Gratuito, simples de usar e permite um bom controle das tasks.
Meet: Gratuito e a conta UFMS permite gravar reuniões.

Ferramentas

Softwares utilizados para produção e gerenciamento de **artefatos**

GitHub: Gratuito e muito importante no gerenciamento de versões.
Drive: A conta UFMS fornece um bom espaço de armazenamento.
Draw.io: Ferramenta mais prática para gerar diagramas e gratuita.

Figma: Uma das melhores ferramentas para design e gratuita.
Miro: Gratuito e prático.

Tecnologias Backend:

.NET: Framework C# em alta no mercado, com um ótimo desempenho e um bom suporte.
Firebase: Facilidade de uso e quantidade de armazenamento de arquivos do plano gratuito suficiente para atender às nossas demandas.
MongoDB: Banco de dados NoSQL e extremamente rápido, o que nos permitiu modificar a estrutura dos registros rapidamente e sem a necessidade de ficar atualizando tabelas (como seria em um banco relacional).

Tecnologias utilizadas para testes:

Postman: A versão gratuita cobriu o que a equipe precisava nos testes de API, e a equipe possuía conhecimento sobre a ferramenta.
Moq: Framework de Mock mais popular e amigável na área de testes unitários com C#.
XUnit: Framework de testes unitários gratuito, voltado para a comunidade é bastante famoso no mundo C#.

Arquitetura C4: Optamos pelo modelo C4 para documentar a arquitetura do sistema. O modelo C4 nos serviu bem pois permitia que todos os integrantes da equipe pudessem ter uma visualização de como os diferentes componentes do sistema se comunicavam em diferentes níveis de abstração, possibilitando que, por exemplo, alguém da equipe do front-end soubesse (em alto nível), como o back-end estava se comportando.

Tecnologia utilizada no aplicativo mobile: Flutter

- Flutter é um framework de código aberto do Google para criar aplicativos multiplataforma bonitos e compilados nativamente a partir de uma única base de código.
- A linguagem de programação utilizada para o desenvolvimento com o Flutter é o Dart, ao qual assim como o framework possui uma boa curva de aprendizagem. O Dart oferece suporte tanto a paradigma funcional, quanto à Programação Orientada à Objetos e possui algumas semelhanças em conceitos e sintaxe as linguagens populares como Kotlin e Javascript, por exemplo;
- O Flutter demonstra ser uma tecnologia confiável quando analisamos algumas das empresas que mantêm projetos com ele, tais como o próprio Google com o Google Play e Stadia, além de Alibaba, Nubank, Ebay, Toyota, Tencent entre outras empresas.
- O Flutter provê recursos que garante alta produtividade e tempo reduzido de desenvolvimento. Dentre os recursos providos, destaca-se: o Hot Reload que reflete as alterações do código na UI quase que instantaneamente sem perda de estado, além de um vasto catálogo de widgets, e mais de 20 mil packages no pub.dev com soluções da UI às funcionalidades de baixo nível como Câmera, Geolocalização, etc.
- Alguns membros do time já tinham experiência trabalhando com essa tecnologia.
- O Flutter possui performance nativa, pois através de sua Dart VM ele compila o projeto escrito em Dart, em binários nativos. Logo, a plataforma rodará a aplicação como seu fosse nativa, dispensando uso de bridge como ocorre no React Native, ou alguma camada intermediária entre a aplicação e a plataforma de execução.
- Contras:
    - sua comunidade não é tão grande quando comparada com a comunidade do desenvolvimento Android Nativo ou do React Native, principal concorrente.
    - A linguagem Dart não é tão popular quanto a linguagens de outras Tecnologias como o Kotlin e o Javascript, e também requer uma certa curva de aprendizagem.

- Funcionalidades específicas que precisem de codificação mais próxima do Hardware, podem requerer conhecimento na linguagem nativa da plataforma, sendo assim, exigir algum conhecimento de programação nativa aos programadores do projeto.

Por que não React Native ?

O Flutter se mostrou melhor na experiência de depuração e na clareza de exibição de erros baseado na experiência dos Devs da equipe. O Flutter aparenta ter recursos mais robustos para depuração e análise de performance, o que pode contribuir na produtividade e tempo desenvolvimento. Mesmo com as vantagens do RN sendo reconhecidas, como o uso do JS e ser fácil de aprender para desenvolvedores que já possuem conhecimento em desenvolvimento web, o Flutter pontuou em pontos considerados mais relevantes.

Por que não Android Nativo ?

O processo de aprendizado de Kotlin com Android nativo no início exige mais tempo, e essa curva de aprendizado foi um ponto negativo contra a escolha de adotar o Android Nativo para o desenvolvimento do projeto. Embora haja pessoas experientes em android nativo na equipe, esse ponto foi suficiente para descartarmos o Android Nativo como opção. Acreditamos que seremos mais produtivos com o Flutter dentro do tempo que temos para o desenvolvimento do projeto deste semestre.

Outro motivo seria a dificuldade de lançar um app para iOS ou outras plataformas caso continuássemos com o desenvolvimento nativo. Como o Flutter possui uma proposta multiplataforma, seria muito mais viável lançar o app para outras plataformas sem precisar codificar em outras linguagens e tecnologias.